

App Dev Project Report

1. Student Details

- **Name:** Rohit Kumar
 - **Roll Number:** 23f1000965
 - **Email:** 23f1000965@ds.study.iitm.ac.in
 - **About Me:** I am a diploma-level student and an enthusiastic learner in web development. I enjoy building practical full-stack applications and improving them through debugging and new feature integration.
-

2. Project Details

- **Project Title:** Hospital Management System

Problem Statement

To design and build a role-based hospital management web application where Admin, Doctor, and Patient can efficiently manage appointments, treatment records, availability, and reporting workflows.

Approach

The project is developed using Flask with modular Blueprints for clean separation of concerns. SQLAlchemy models are used for relational data handling, and the frontend is built with HTML/CSS/JavaScript. The application supports role-based authentication, appointment lifecycle management, treatment tracking, async background jobs, CSV export, and performance optimization through caching.

3. AI/LLM Declaration

I used ChatGPT assistance for selected tasks such as API shaping suggestions, documentation drafting. The estimated AI/LLM usage is moderate (around 10-15%) and for productivity support. Final integration decisions, debugging, verification, and project-level implementation were performed manually.

4. Technologies and Frameworks Used

Technology / Library Purpose

Flask - Core backend web framework

Flask Blueprints - Modular route organization (auth/admin/doctor/patient)

Flask-JWT-Extended - Authentication and role-protected APIs

Flask - CORS - Cross-origin API access for frontend

Flask - Caching Response caching for performance

SQLAlchemy - ORM for relational database operations

SQLite - Primary local database

Celery - Background task processing

Redis Celery - result backend and cache backend

MailHog (SMTP + UI) - Local email testing for reminders

HTML/CSS/JavaScript - Frontend implementation

Bootstrap (modal/layout usage) - UI components and responsive layout patterns

5. Database Schema

Main Tables

- **User:** Common user identity, credentials, role, active status
- **DoctorProfile:** Doctor-specific data (department, license)
- **PatientProfile:** Patient-specific data (DOB, contact details)
- **Department:** Department master data
- **Appointment:** Booking records with status (BOOKED, COMPLETED, CANCELLED)
- **Treatment:** Diagnosis, prescription, medicines against appointments
- **DoctorAvailability:** Doctor date/slot availability

Relationships

- One-to-One: User -> DoctorProfile / PatientProfile
 - One-to-Many: Department -> DoctorProfile
 - One-to-Many: DoctorProfile -> Appointment
 - One-to-Many: PatientProfile -> Appointment
 - One-to-Many: Appointment -> Treatment
 - One-to-Many: DoctorProfile -> DoctorAvailability
-

6. API Resource Endpoints (Representative)

Auth APIs

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/profile

Admin APIs

- GET /api/admin/dashboard
- GET/POST /api/admin/doctors
- GET/PUT/DELETE /api/admin/doctors/<doctor_id>
- GET /api/admin/patients
- DELETE /api/admin/patients/<patient_id>
- GET /api/admin/appointments
- PUT /api/admin/appointments/<appt_id>
- GET /api/admin/appointments/<appt_id>/detail
- GET/POST /api/admin/departments
- PUT/DELETE /api/admin/departments/<dept_id>

Doctor APIs

- GET /api/doctor/dashboard
- GET /api/doctor/patients
- PUT /api/doctor/appointments/<appt_id>
- GET /api/doctor/appointments/<appt_id>/treatment
- GET /api/doctor/appointments/<appt_id>/patient-history
- GET/PUT /api/doctor/profile
- GET /api/doctor/notifications
- GET/PUT /api/doctor/availability

Patient APIs

- GET /api/patient/dashboard
- GET /api/patient/departments
- GET /api/patient/departments/<department_id>
- GET /api/patient/doctors
- GET /api/patient/doctors/<doctor_id>
- GET /api/patient/doctors/<doctor_id>/availability
- POST/GET /api/patient/appointments

- PUT /api/patient/appointments/<appointment_id>/cancel
- GET /api/patient/history
- GET/PUT /api/patient/profile
- POST /api/patient/exports/history-csv
- GET /api/patient/exports/history-csv/<task_id>
- GET /api/patient/exports/history-csv/<task_id>/download

YAML API Definition File:

Included separately in the submission ZIP as api.yaml.

7. Architecture and Implemented Features

Architecture Overview

- Backend/run.py - application entry point
 - Backend/app/app.py - app factory, config, extension initialization
 - Backend/app/models.py - SQLAlchemy models and relationships
 - Backend/app/routes/ - role-based API modules
 - Backend/celery_app.py - Celery app and beat scheduler setup
 - Backend/tasks.py - background jobs (daily reminder, monthly report, CSV export)
 - Frontend/index.html + Frontend/JS - role dashboards and UI interactions
-

8.Video Presentation

Drive link:-

https://drive.google.com/file/d/1wN2JItFmwCOcnBydQXMjp_2PLlknmp4J/view?usp=sharing